Technical Interview Notes & Cheat Sheet

Quick reference guide to explain every architectural decision, data flow, and file in plain English during an interview.

1. Folder Structure Explained Simply

- `src/components/common/` : Reusable UI elements used across the entire app (Modal, ProgressBar, Header, EmptyState, Loader, ErrorState, ProtectedRoute).
- `src/components/student/` : Components built specifically for student assignment viewing and submissions (AssignmentCard, StudentProgress).
- `src/components/admin/` : Components built specifically for professor assignment authoring and tracking (AssignmentForm, AdminAssignmentCard, StudentStatusList).
- `src/context/` : React Context providers sharing global state across the app without prop drilling (AuthContext, DataContext, ThemeContext).
- `src/hooks/` : Custom hooks encapsulating reusable stateful logic (`useLocalStorage` for persistent reactive storage).
- `src/data/` : Initial seed data (2 professors, 5 students, 8 assignments, pre-filled submissions).
- `src/pages/` : Top-level page views rendered by React Router (LoginPage, StudentDashboard, AdminDashboard).
- `src/utils/` : Shared pure helpers (`formatDate` , `isOverdue` , `getPercentage` , `isValidUrl`) and app constants (`ROLES` , `STATUS` , `STORAGE_KEYS`).
- `src/test/` : Automated unit and integration test suite using Vitest and React Testing Library.

---

2. File-by-File Summary

Core

- `src/main.jsx` : Application entry point that mounts React into the HTML root element.
- `src/App.jsx` : Sets up client-side routes (`/` , `/student` , `/admin`), role-based protection, and wraps providers.
- `src/index.css` : Injects Tailwind CSS directives (`base` , `components` , `utilities`).

Utils & Data

- `src/utils/constants.js` : Central repository for constants (`ROLES` , `STATUS` , `STORAGE_KEYS` , `LOADING_DELAY`) to prevent magic strings.
- `src/utils/helpers.js` : Pure utility functions for date formatting, safe percentage calculation, overdue detection, and HTTP URL validation.
- `src/data/seedData.js` : Realistic seed data for users, assignments, and submissions when local storage is empty.

Hooks & Context

- `src/hooks/useLocalStorage.js` : Custom hook that reads state on mount, auto-saves changes to localStorage, and syncs across open browser tabs.
- `src/context/AuthContext.jsx` : Manages the active authenticated user, validates role schemas, and provides login/logout actions.

- `src/context/DataContext.jsx` : Central store for assignments and submissions with helper methods for CRUD operations and student status queries.
- `src/context/ThemeContext.jsx` : Manages light and dark mode toggling by syncing a `.dark` class on the root HTML element.

Common Components

- `src/components/common/Header.jsx` : Navigation bar with brand logo, user badge, theme toggle, and logout button.
- `src/components/common/ProtectedRoute.jsx` : Route guard redirecting unauthenticated users to `/` and enforcing role permissions.
- `src/components/common/Modal.jsx` : Accessible dialog overlay that traps keyboard focus, closes on Escape/backdrop click, and restores focus on close.
- `src/components/common/ProgressBar.jsx` : Visual progress bar with clamped percentages and zero-division protection.
- `src/components/common/EmptyState.jsx` : Friendly placeholder card with optional action button when no data exists.
- `src/components/common/Loader.jsx` : Centered animated spinner displayed during brief simulated loading delays.
- `src/components/common/ErrorState.jsx` : Storage error screen with a "Reset data" button to restore seed data if corrupted.

Student Components

- `src/components/student/AssignmentCard.jsx` : Displays assignment title, description, formatted due date, overdue badge, Drive link, and submit trigger.
- `src/components/student/StudentProgress.jsx` : Summarizes personal student progress (e.g. "3 of 5 submitted") with a progress bar.

Admin Components

- `src/components/admin/AdminAssignmentCard.jsx` : Displays professor assignment card with Edit, Delete, and embedded student tracking.
- `src/components/admin/AssignmentForm.jsx` : Modal form for creating/editing assignments with field validation, Drive link check, and student checkboxes.
- `src/components/admin/StudentStatusList.jsx` : Lists enrolled students with live status badges and class completion percentage.

Pages

- `src/pages/LoginPage.jsx` : Mock user switcher grouped into Admins and Students for instant sign-in.
- `src/pages/StudentDashboard.jsx` : Main student view with progress overview, status filter tabs, due date sorting, and assignment grid.
- `src/pages/AdminDashboard.jsx` : Main professor view for creating, editing, and deleting assignments and tracking class progress.

---

3. The Full Data Flow (Step-by-Step)

```
[1. User clicks profile on LoginPage]
              ↓
[2. AuthContext sets currentUser in localStorage]
              ↓
[3. React Router navigates to /student or /admin]
              ↓
[4. ProtectedRoute verifies user.role matches route]
              ↓
[5. Dashboard reads assignments from DataContext]
              ↓
[6. Student clicks "Yes, I have submitted" -> Modal -> Confirm]
              ↓
[7. DataContext.submitAssignment updates submissions state & localStorage]
              ↓
[8. React re-renders: Student badge turns green, Progress increments]
              ↓
[9. Admin views assignment card: DataContext calculates matching submissions live]
```

---

4. Why These Architectural Decisions? (Say Out Loud)

1. Why separate `submissions` from `assignments` instead of nesting submissions inside each assignment?

> *"In relational and real-world database design, assignments and students have a many-to-many relationship. If 5 students are assigned one project, each student has an independent submission status and timestamp. Storing submissions as a separate array `[{ assignmentId, studentId, status, submittedAt }]` prevents concurrent state conflicts, mirrors relational best practices, and makes querying and cascade deletion clean."*

2. Why Context API instead of Redux or Zustand?

> *"For an application of this scope with two primary entities (auth and assignments), React Context API provides clean, predictable global state without adding external library overhead, boilerplate code, or increasing bundle size."*

3. Why `localStorage` with a custom hook?

> *"Since there is no backend server, localStorage enables data persistence across reloads. The custom `useLocalStorage` hook makes localStorage reactive in React, handles JSON serialization, recovers from corrupt data with fallback defaults, and synchronizes across open tabs."*

4. Why React Router with ProtectedRoute?

> *"React Router provides declarative client-side routing. The custom ProtectedRoute component ensures role-based authorization so students cannot access admin routes and vice versa, redirecting unauthorized users cleanly."*

---

5. 10 Likely Interview Questions with Simple Answers

Q1: How did you prevent duplicate submissions if a student double-clicks "Confirm"?

Answer: In `DataContext.submitAssignment`, we perform an existence check (`prev.some(...)`) before appending. If a submission for that assignment and student already exists, state is returned unchanged. The confirm button is also disabled while submitting.

Q2: How do you handle division by zero in your progress bar?

Answer: In `getPercentage(value, total)`, we check `if (!total || total <= 0) return 0`. This guarantees that if 0 students are assigned or a student has 0 assignments, the component renders `0%` cleanly without producing `NaN`.

Q3: How do you prevent timezone bugs when calculating overdue assignments?

Answer: We compare standardized local calendar date strings formatted as `YYYY-MM-DD` (e.g. `'2026-10-15' < '2026-11-04'`). Lexicographical string comparison on ISO dates is 100% timezone-safe and avoids UTC vs local midnight offsets. If an assignment is due today, it is not marked overdue until tomorrow.

Q4: How is data isolation enforced between users?

Answer: In the student dashboard, we filter `assignments.filter(a => a.assignedTo.includes(studentId))`. In the admin dashboard, we filter `assignments.filter(a => a.createdBy === adminId)`. DataContext mutations (`updateAssignment`, `deleteAssignment`) also verify creator ownership.

Q5: What happens if a user's localStorage contains corrupted or invalid JSON?

Answer: `useLocalStorage` wraps `JSON.parse` in a `try/catch` block. If parsing throws, it logs a warning, falls back to seed data, and invokes an `onError` callback that displays an ErrorState banner with a "Reset data" button.

Q6: How did you ensure accessibility (a11y) in the Modal component?

Answer: The modal has `role="dialog"`, `aria-modal="true"`, and `aria-labelledby`. On mount, it focuses the first interactive element and traps Tab/Shift+Tab focus inside the dialog. When closed, it automatically restores focus back to the triggering element.

Q7: Why didn't you store filtered lists and percentage progress in React useState?

Answer: Storing derived data in state creates redundant state and synchronization bugs. Instead, we calculate filtered lists (`visibleAssignments`) and progress counts as plain derived variables directly during render from existing state.

Q8: How did you protect against XSS when students open Drive links?

Answer: We validate all links with `isValidUrl` ensuring the protocol is strictly `http:` or `https:` , which rejects dangerous protocols like `javascript:alert(1)` . When rendering links, we use `target="_blank"` with `rel="noopener noreferrer"` .

Q9: What happens to submissions when an admin edits an assignment and removes a student?

Answer: In `updateAssignment` , whenever `assignedTo` is modified, we filter the submissions list to prune any records where `assignmentId` matches and the student is no longer in the new `assignedTo` array. This prevents orphaned submission records.

Q10: How does multi-tab synchronization work?

Answer: In `useLocalStorage` , we attach a listener to `window.addEventListener('storage', ...)` . When storage changes in another browser tab, the event listener parses the new value and updates React state so all open tabs stay synchronized without a refresh.

---

6. Known Limitations & Future Improvements

- **Client-Side Storage Quota**: `localStorage` has a ~5MB quota and is scoped to a single browser. In production, this would be backed by a REST/GraphQL API and PostgreSQL database.
- **Mock Authentication**: There is no password verification or JWT session token. In production, we would integrate OAuth2, Auth0, or Supabase Auth.
- **Real File Attachments**: The system records submission confirmation timestamps rather than uploading binary PDF/ZIP files. With more time, we would integrate AWS S3 or Cloudinary for direct file uploads.